

From RNN Attention to Transformers

Self-Attention, Positional Encoding, and Modern Machine Translation

Where we are coming from

In the previous lecture:

- Vanilla seq2seq RNN translation used one fixed context vector
- Attention improved this by giving the decoder access to all encoder states
- But even with attention, the model was still recurrent

Remaining problems:

- sequential computation
- slow training
- slow inference
- difficulty with very long-range dependencies

This leads to the next question:

If attention is the key idea, do we really need recurrence at all?

A new principle: every token can look at every other token

Suppose we have token representations:

$$x_1, x_2, \dots, x_T$$

For each token, we want to build a new representation by looking at other relevant tokens in the sequence.

For token i , we would like:

$$z_i = \sum_{j=1}^T \alpha_{i,j} \text{ information from token } j$$

where:

- $\alpha_{i,j}$ says how much token i should attend to token j
- the weights depend on the content
- each token gets its own weighted summary

This is the core idea of **self-attention**.

From attention in seq2seq to self-attention

In RNN translation with attention:

- decoder state queried encoder states
- this was **cross-attention** in spirit

In self-attention:

- each token queries the other tokens in the *same* sequence
- source tokens attend to source tokens
- target tokens attend to earlier target tokens

So the progression is:

encoder-decoder attention $\longrightarrow$ self-attention within a sequence

This lets the model build context-rich token representations without recurrence.

Queries, keys, and values

Self-attention uses three learned projections for each token representation x_i :

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

Interpretation:

- Query: what this token is looking for
- Key: what this token offers for matching
- Value: the information this token contributes if selected

The similarity between token i and token j is measured by:

$$q_i^\top k_j$$

Then token i forms a weighted combination of the values v_j .

Self-attention equations

For token i , compute scores with all tokens j :

$$e_{i,j} = q_i^\top k_j$$

Scale and normalize:

$$\alpha_{i,j} = \frac{\exp\left(\frac{q_i^\top k_j}{\sqrt{d_k}}\right)}{\sum_{m=1}^T \exp\left(\frac{q_i^\top k_m}{\sqrt{d_k}}\right)}$$

Then compute the output representation:

$$z_i = \sum_{j=1}^T \alpha_{i,j} v_j$$

Important point:

- every token can aggregate information from all other tokens
- relevant tokens get larger weights

Matrix form of self-attention

Collect all token representations row-wise into a matrix:

$$X \in \mathbb{R}^{T \times d}$$

Then:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Self-attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- all pairwise token interactions are computed together
- highly parallelizable on modern hardware
- no recurrent chain across time

Intuition: why self-attention helps

Consider the sentence:

The animal didn't cross the street because it was tired.

To understand the word *it*, the model may need to relate it to *animal*.

In an RNN:

- this information travels through many recurrent steps

In self-attention:

- the token *it* can directly attend to *animal*

So the path between distant tokens becomes much shorter.

RNN path length grows with distance vs self-attention path length is direct

Order of Tokens

A pure attention mechanism does not inherently know token order.
Without extra information, these sequences would look similar as bags of tokens:

(dog, bites, man) (man, bites, dog)

So Transformers need an explicit way to represent position.

Solution: positional encoding

For each token representation, add a position-dependent vector:

$$x_i^{(\text{input})} = e_i + p_i$$

where:

- e_i is token embedding
- p_i is positional encoding

Sinusoidal positional encoding

A famous choice is sinusoidal positional encoding:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Why this is useful:

- gives each position a distinct pattern
- lets the model infer relative positions from combinations of sine and cosine values
- does not require recurrence

In practice, modern models may use learned or relative positional representations, but the core idea is the same:

order must be injected explicitly

What is a Transformer encoder block?

A Transformer encoder block typically contains:

- 1 Multi-head self-attention
- 2 Add & LayerNorm
- 3 Position-wise feedforward network
- 4 Add & LayerNorm

Symbolically:

$$X \rightarrow \text{Self-Attention} \rightarrow \text{Add\&Norm} \rightarrow \text{FFN} \rightarrow \text{Add\&Norm}$$

Role of the encoder:

- build contextualized representations of source tokens
- each source token becomes aware of relevant other source tokens

What is a Transformer decoder block?

A Transformer decoder block has three main parts:

- 1 Masked self-attention over previously generated target tokens
- 2 Cross-attention to encoder outputs
- 3 Feedforward network

Why masked self-attention?

- when predicting target token y_t , the model must not look at future target tokens

So attention is masked to allow only:

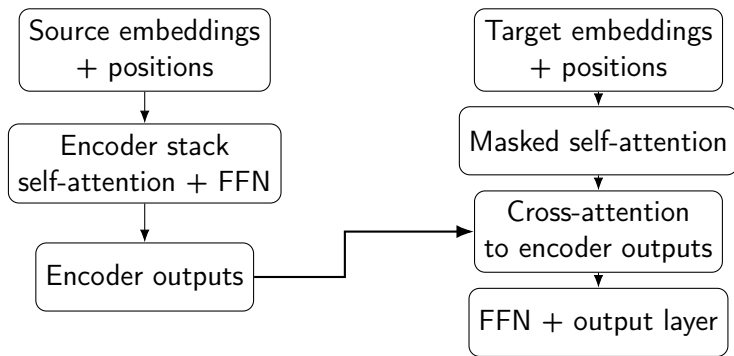
$$y_1, \dots, y_{t-1}$$

when predicting y_t .

Cross-attention:

- target-side states query the encoder outputs

Diagram of the Transformer for translation



The decoder now uses:

- target-side masked self-attention
- encoder–decoder cross-attention

Why multi-head attention?

A single attention mechanism produces one weighted view of the sequence.
Multi-head attention allows several attention patterns in parallel:

$$\text{head}_r = \text{Attention}(Q_r, K_r, V_r)$$

Then combine them:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$$

Intuition:

- one head may focus on local structure
- another may focus on long-range dependencies
- another may capture syntactic relations

So multi-head attention gives multiple learned views of relevance.

Comparison: RNN attention vs Transformer

	RNN + Attention	Transformer
Sequence processing	Sequential	Parallel across positions
Main mechanism	Recurrence + attention	Self-attention + cross-attention
Long-range interaction	Improved but indirect	More direct
Order information	Comes from recurrence	Added explicitly by position encoding
Training speed	Slower	Faster on modern hardware
Scalability	Limited	Stronger

Summary

- Attention suggested a deeper principle: keep representations available and retrieve what is relevant
- Transformers apply this principle directly within sequences using self-attention
- Queries, keys, and values define content-based interaction
- Positional encoding supplies order without recurrence
- Decoder uses masked self-attention and cross-attention
- This removes the main limitations of RNN-based translation

RNN memory → attention retrieval → Transformer